

Deadline-Aware Elastic Provisioning for Hibernation-Tolerant Bag-of-Tasks Scheduling on Spot and Burstable Cloud Instances

Abstract—Spot virtual machines cut cloud compute prices substantially but may be hibernated by the provider at any time, which threatens deadline-constrained workloads. Burst-HADS tolerates such interruptions by migrating the tasks of a hibernated instance onto burstable VMs running in burst mode, falling back to on-demand capacity when no burstable instance is free. Because a burstable instance accepts only one task at a time and the number provisioned is a fixed fraction of the selected fleet, the rescue capacity available at the moment of an interruption is bounded independently of how much work is actually displaced. We present R-BurstHADS, which extends Burst-HADS’s dynamic scheduler with elastic spot provisioning drawn from the same instance catalogue, sized by a work-conservation argument against the remaining deadline rather than by a fixed queue-length constant. Across 2,700 simulation runs spanning five hibernation scenarios, six workload sizes and 30 seeds, R-BurstHADS reduces makespan relative to Burst-HADS by 26.0–39.8% in every scenario, while its monetary-cost advantage scales monotonically with the interruption rate: it is cost-neutral at $k_h = 1$, 12.5% cheaper at $k_h = 3$, and 14.3–64.3% cheaper at $k_h = 5$. All three schedulers complete 100% of tasks, but only the two burstable-aware schedulers meet the deadline in every configuration. We first validate our HADS and Burst-HADS implementations against the published results of the source work, reproducing the reported ordering and the direction and magnitude of its cost differences.

Index Terms—cloud computing, spot instances, burstable instances, task scheduling, hibernation, bag-of-tasks, deadline constraints

I. INTRODUCTION

Infrastructure-as-a-Service providers sell surplus capacity at a steep discount under names such as Amazon EC2 Spot. The discount is paid for in reliability: the provider may reclaim the instance at short notice. Amazon EC2 additionally offers *hibernation*, in which the instance’s memory and context are written to its EBS root volume and execution is suspended rather than destroyed, with the instance resuming when capacity becomes available again. For a workload with a deadline, hibernation is not free even though the checkpoint preserves progress: the capacity disappears for an unknown period, and the displaced work must be placed somewhere else in time to finish.

Bag-of-Tasks (BoT) applications — parameter sweeps, Monte Carlo simulation, chromosome mapping, image processing — are a natural fit for this market. Their tasks are independent, so a lost instance costs only the work resident

on it, and the remaining tasks can in principle be redistributed freely.

Teylo et al. addressed this setting in two stages. HADS [1] schedules a BoT application across spot and non-burstable on-demand instances and, when a spot VM hibernates, migrates its unfinished tasks to other running instances or to newly launched on-demand ones. Burst-HADS [2] extends this with *burstable* instances, which accumulate CPU credits while idle and can then be driven at full processing power in burst mode; a hibernation is answered first by migrating a task to an idle burstable VM, and only then by falling back to on-demand capacity.

The rescue capacity Burst-HADS can call on at the instant of an interruption is, however, fixed in advance. Its burstable fleet is sized once, after the primary scheduling step, as a fixed fraction of the instances the search selected, and a burstable instance holds only one task at a time so that credits continue to accrue. Neither quantity depends on how much work a hibernation actually displaces or on how much deadline slack remains when it occurs. When the displaced work exceeds the standing burstable capacity, every remaining task falls through to on-demand instances, which are the most expensive resource in the catalogue and carry a start-up latency.

This paper makes the following contributions.

- We present **R-BurstHADS**, which adds two provisioning behaviours to Burst-HADS: a risk-triggered *preemptive* step (Algorithm 4) that launches a replacement spot instance for a high-risk VM before it is interrupted, and a *deadline-aware reactive* step (Algorithm 5) that provisions the minimum additional capacity required for the displaced work to finish before the deadline. Both draw from the same instance catalogue available to the baselines.
- We **validate our baseline implementations** against the published results of Burst-HADS before using them for comparison (Section VI).
- We evaluate all three schedulers under the **hibernation scenarios of the source work** across six workload sizes and 30 seeds, and show that R-BurstHADS’s cost advantage grows monotonically with the interruption rate while its makespan advantage is essentially rate-independent (Section VII).

II. BACKGROUND AND RELATED WORK

A. Scheduling on interruptible instances

Work on interruptible cloud instances divides broadly into bidding and checkpointing strategies for surviving reclamation, and scheduling strategies that treat interruption as a first-class constraint. AutoBoT-like approaches select instance types under a deadline and budget but do not model hibernation. The line we build on schedules explicitly against a hibernation process.

B. HADS and Burst-HADS

HADS [1] formulates BoT scheduling on spot and on-demand instances as a cost-minimisation problem under a deadline D , building a static map with a greedy single-pass construction and migrating tasks away from hibernated instances at runtime. Burst-HADS [2] replaces the greedy construction with an Iterated Local Search (ILS), adds burstable instances as a rescue tier, and introduces a work-stealing procedure that rebalances load whenever an instance becomes idle.

Because Burst-HADS's Burst Migration Procedure draws burstable candidates from the idle set and moves each recipient to the busy set on assignment, a burstable instance accepts exactly one migrated task at a time. Combined with a fleet size fixed at scheduling time, this bounds the burstable rescue capacity independently of the size of the interruption — the gap this work addresses.

III. SYSTEM MODEL AND PROBLEM FORMULATION

An application is a set T of $|T|$ independent tasks with a common deadline D . A task t_i has execution time e_i on a reference core and memory requirement m_i . The provider offers a catalogue M of VM types; vm_j has processing speed s_j , virtual core count nvc_j , memory M_j and price c_j per unit time. A multi-core instance executes up to nvc_j tasks simultaneously, one per core, so its whole-instance throughput is $s_j \cdot nvc_j$. Symbols are collected in Table I.

Cost is instance-seconds: an instance is charged from launch until termination or hibernation, whether or not it is executing. The objective is

$$\min \alpha \cdot \widehat{cost} + (1 - \alpha) \cdot \widehat{mkp} \quad \text{s.t.} \quad mkp \leq D \quad (1)$$

over normalised cost and makespan. Candidate instances are ranked by

$$weight(vm_j) = \frac{Gflops_j}{c_j} = \frac{s_j \cdot nvc_j}{c_j} \quad (2)$$

where $Gflops_j$ quantifies the whole instance's computing power. Placement on spot instances is constrained not by D but by

$$D_{spot} = D - \max_i \frac{e_i \cdot (1 + \omega)}{s_{min}} \quad (3)$$

a reserve guaranteeing that the longest task can still be migrated to the slowest instance and executed before D if its host hibernates.

TABLE I
VARIABLES AND PARAMETERS USED IN THE ALGORITHMS

Symbol	Meaning
T, t_i	task set; task i
e_i, m_i	execution time and memory requirement of t_i
e_{ij}	execution time of t_i on vm_j
M, vm_j	VM catalogue; instance j
s_j, nvc_j	speed and virtual-core count of vm_j
M_j, c_j	memory capacity and price per period of vm_j
$\Theta(\cdot)$	throughput of a VM set, $\sum_j s_j \cdot nvc_j$
D	application deadline
D_{spot}	deadline minus the worst-case migration reserve
RD_{spot}	relaxed D_{spot} used inside the ILS
ω	checkpoint overhead fraction
T_{start}	instance start-up latency
cc_j, rcc_{ij}	accrued and required CPU credits
λ_h, λ_r	hibernation and resumption rates
k_h, k_r	expected hibernations / resumptions over D
S, S_{best}	candidate and incumbent scheduling solutions
α	cost/makespan weight in the fitness function
$burst_rate$	burstable fleet size as a fraction of $ S $
Q_l	unfinished tasks of a hibernated instance vm_l
IR, BR	sets of idle and busy instances
M^o, M^b	elastic on-demand and burstable pools
A	active replacement instances provisioned at runtime
W	remaining work of the tasks awaiting placement
ρ	preemptive provisioning risk threshold
p_v	hibernation probability of vm_v over one task

IV. R-BURSTHADS

R-BurstHADS inherits Burst-HADS's ILS (Algorithm 1), greedy construction (Algorithm 2), local search (Algorithm 3), burst migration (Algorithm 6) and work stealing (Algorithm 7), and adds two provisioning behaviours, Algorithms 4 and 5. Both launch instances of the most cost-efficient spot type present in the scheduler's own pool, $\arg \max_j s_j nvc_j / c_j$, so the proposed method has access to no instance type the baselines cannot also select.

A. Preemptive replacement provisioning

Let p_v be the probability that spot instance v hibernates during the execution of an average task, derived from its rate λ_h . For each v with p_v above a threshold ρ , R-BurstHADS compares the expected cost of rescuing v 's tasks reactively against the cost of holding a replacement ready, and provisions only when the comparison favours it (Algorithm 4).

B. Deadline-aware capacity sizing

When spot capacity is lost, R-BurstHADS sizes the replacement fleet from the deadline rather than from a fixed queue-length target. With W the remaining work of the tasks awaiting placement, t the current time and $\Theta(\cdot)$ the throughput of an instance set,

$$T = D - t - T_{start}, \quad n_{extra} = \left\lceil \frac{W/T - \Theta(A)}{\theta} \right\rceil \quad (4)$$

where A is the set of active replacements and θ the throughput of one new instance. If the surviving fleet can already retire the queue before D the deficit is non-positive and nothing

Algorithm 1 R-BurstHADS primary scheduler

```
1:  $D_{spot} \leftarrow \text{Eq. (3)}$ 
2:  $S \leftarrow \text{GREEDYINITIAL}(T, M, D_{spot})$ 
3:  $S_{best} \leftarrow \text{LOCALSEARCH}(S, D_{spot})$ 
4:  $RD_{spot} \leftarrow D_{spot}$ ;  $it_b \leftarrow 0$ ;  $it_r \leftarrow 0$ 
5: for  $it = 1$  to  $max\_iteration$  do
6:   if unused spot instances remain then
7:     add one at random to  $S.selected$ 
8:   end if
9:   if  $it - it_b > max\_failed$  and  $it - it_r > max\_failed$  then
10:     $RD_{spot} \leftarrow \min(RD_{spot}(1 + relaxed\_rate), D)$ 
11:     $it_r \leftarrow it$ 
12:   end if
13:    $S' \leftarrow \text{LOCALSEARCH}(S, RD_{spot})$ 
14:   if  $fitness(S') < fitness(S_{best})$  then
15:     $S_{best} \leftarrow S'$ ;  $it_b \leftarrow it$ 
16:   end if
17:    $S \leftarrow S'$ 
18: end for
19:  $n_b \leftarrow \lceil burst\_rate \cdot |S_{best}.selected| \rceil$ 
20:  $B \leftarrow \text{LAUNCHBURSTABLE}(n_b)$  from  $M^b$ 
21: for all  $t_i$  with predicted finish  $> D_{spot}$ , latest first do
22:    $b \leftarrow$  next free instance of  $B$ 
23:   if  $b \neq \emptyset$  and baseline finish of  $t_i$  on  $b$  improves on its current finish then
24:     move  $t_i$  to  $b$  in baseline mode
25:   end if
26: end for
27:  $\text{PREEMPTIVEPROVISION}(S_{best})$  {Alg. 4}
28: return  $S_{best}$ 
```

is provisioned; the scheduler then behaves exactly as BurstHADS. Should the estimate prove optimistic, the per-task placement path adds capacity incrementally under its own deadline test, so no safety margin is applied to (4).

V. EXPERIMENTAL SETUP

A. Simulator, catalogue and workload

We use a discrete-event simulator in which each instance executes up to nvc_j tasks concurrently. Hibernation stops the charge and checkpoints resident tasks; a resumed instance continues from its checkpoint. The shared pool (Table II) contains three instances of each spot type, one burstable instance seeding the elastic pool M^b , and one on-demand instance as the template for the unbounded pool M^o . Prices are us-east-1 rates. Per-instance-type hibernation rates use the source work's worst-case parameterisation ($k_h = 5$ over $D = 2700$ s) for m5.xlarge and a low nominal rate for the remaining spot types; this differentiation is a modelling choice of the present study rather than a measured quantity.

Tasks are drawn independently with execution time $\mathcal{U}[100, 350]$ s and memory $\mathcal{U}[2.85, 13.19]$ MB, matching the ranges reported for the reference jobs, with $|T| \in \{10, 20, 50, 100, 200, 300\}$.

Algorithm 2 GREEDYINITIAL: initial solution

```
1: sort  $T$  by memory requirement, descending
2: for all  $t_i \in T$  do
3:   for all  $vm_j \in selected$ , least loaded first do
4:     if  $\text{FEASIBLE}(t_i, vm_j, D_{spot})$  then
5:       assign  $t_i$  to  $vm_j$ ; continue with next task
6:     end if
7:   end for
8:    $K \leftarrow \{vm_j \in M^s \setminus selected : \text{FEASIBLE}(t_i, vm_j, D_{spot})\}$ 
9:    $vm_j \leftarrow \text{WRRPICK}(K)$  using Eq. (2)
10:  if  $vm_j \neq \emptyset$  then
11:    select and assign; continue with next task
12:  end if
13:  for all  $vm_j \in M^o$  already running, cheapest first do
14:    if  $\text{FEASIBLE}(t_i, vm_j, D)$  then
15:      assign; continue with next task
16:    end if
17:  end for
18:   $vm_{new} \leftarrow \text{LAUNCHONDEMAND}()$  from  $M^o$ 
19:  if  $\text{FEASIBLE}(t_i, vm_{new}, D)$  then
20:    assign
21:  else
22:    abort: no feasible placement before  $D$ 
23:  end if
24: end for
```

Algorithm 3 LOCALSEARCH

Require: solution S , bound d

```
1:  $best \leftarrow S$ 
2:  $dest \leftarrow$  one instance chosen at random from  $S.selected \cup$  unused spot
3: for  $a = 1$  to  $max\_attempt$  do
4:    $cand \leftarrow clone(best)$ 
5:    $R \leftarrow$  random sample of  $swap\_rate \cdot |T|$  tasks
6:   for all  $t_i \in R$  do
7:     if memory of  $dest$  permits then
8:       move  $t_i$  to  $dest$ 
9:     else
10:      revert  $t_i$ 
11:    end if
12:   end for
13:   if  $fitness(cand, d) < fitness(best, d)$  then
14:      $best \leftarrow cand$ 
15:   end if
16: end for
17: return  $best$ 
```

B. Deadline derivation

The deadline is derived from the pool rather than fixed, so that experimental difficulty is invariant to pool size:

$$M^* = \max\left(\frac{|T|\bar{e}}{\Theta(M^s)}, \frac{e_{\max}}{s_{\max}}\right), \quad D = DF \cdot \kappa \cdot M^* \quad (5)$$

Algorithm 4 PREEMPTIVEPROVISION (Theorem 1)

Require: incumbent S_{best} , threshold ρ

```

1:  $\bar{e} \leftarrow$  mean execution time over  $T$ 
2:  $r_s, s_s \leftarrow$  price and speed of  $\arg \max_j s_j nvc_j / c_j$  over  $M^s$ 
3: for all  $vm_v \in M^s$  with  $\lambda_h(v) > 0$  do
4:    $p_v \leftarrow 1 - e^{-\lambda_h(v) \bar{e} / s_v}$ 
5:   if  $p_v < \rho$  then
6:     continue
7:   end if
8:    $n_v \leftarrow |\{t_i : S_{best}(t_i) = v\}|$ 
9:    $C_{react} \leftarrow n_v \bar{e} (r_o / s_o - r_s / s_s)$ 
10:   $C_{proact} \leftarrow T_{start} r_s + (\bar{e} / s_s) r_s$ 
11:  if  $p_v \cdot C_{react} > C_{proact}$  then
12:    launch one replacement, ready at  $T_{start}$ 
13:  end if
14: end for

```

Algorithm 5 DEADLINEAWAREPROVISION (Theorem 2)**Require:** queued tasks Q , current time t , active replacements A

```

1:  $W \leftarrow \sum_{t_i \in Q}$  remaining time of  $t_i$ 
2:  $T \leftarrow D - t - T_{start}$ 
3: if  $W \leq 0$  or  $T \leq 0$  then
4:   return 0
5: end if
6:  $\theta \leftarrow s_{new} \cdot nvc_{new}$ 
7:  $deficit \leftarrow W / T - \Theta(A)$ 
8: if  $deficit \leq 0$  then
9:   return 0
10: end if
11:  $n \leftarrow \min(max\_per\_event, \lceil deficit / \theta \rceil)$ 
12: launch  $n$  instances, ready at  $t + T_{start}$ 
13: redistribute  $Q$  over  $A \cup new$  by earliest estimated finish
14: return  $n$ 

```

with $\kappa = 3$ calibrated so that $DF = 1$ is met by a well-packed undisturbed run but not by a disrupted one without escalation. A floor keeps D above the migration reserve of (3), below which the problem is infeasible rather than tight. All reported experiments use $DF = 1$.

C. Hibernation scenarios and parameters

Interruptions follow the source work's scenarios (Table III), in which each spot instance is subject to an independent Poisson process with rate $\lambda_h = k_h / D$ and, where resumption is modelled, $\lambda_r = k_r / D$. Algorithm parameters are given in Table IV. Every configuration is run with 30 seeds; all three schedulers see a byte-identical task set and interruption stream for a given seed, so the comparison is paired at the level of the workload. In total the evaluation comprises 2,700 simulation runs.

VI. BASELINE VALIDATION

A comparison is only as trustworthy as its baseline implementation. We therefore first reproduce the published compari-

Algorithm 6 Burst migration procedure

Require: Q_l, IR, BR, M^o, D

```

1:  $Q_l \leftarrow \text{SORTTASKS}(Q_l)$  {checkpointed tasks first}
2: for all  $t_i \in Q_l$  do
3:   Attempt 0 (R-BurstHADS): a provisioned replacement in  $A$  with free capacity that finishes  $t_i$  before  $D$ 
4:   Attempt 1: burstable  $vm_j \in IR$  with  $cc_j > rcc_{ij}$ ; migrate in burst mode,  $IR \leftarrow IR \setminus \{vm_j\}$ ,  $BR \leftarrow BR \cup \{vm_j\}$ 
5:   Attempt 2: non-burstable  $vm_j \in \text{SORTBYMARKET}(IR \cup BR)$  passing  $\text{CHECKMIGRATION}(t_i, vm_j, D)$ 
6:   Attempt 3 (R-BurstHADS): DEADLINEAWAREPROVISION {Alg. 5}
7:   Attempt 4: new  $vm_j \in M^o$ , cheapest first, if  $start_{ij} + e_{ij} + \omega < D$ 
8: end for

```

Algorithm 7 Work stealing

Require: newly idle vm_k , busy set BR , deadline D

```

1: for all non-burstable  $vm_j \in BR$ , most loaded first do
2:   for all waiting  $t_i$  on  $vm_j$  do
3:     if  $\text{SLACK}(t_i, vm_k, D) > 0$  then
4:       move  $t_i$  to  $vm_k$ 
5:       if  $vm_k$  is burstable then
6:         mark  $t_i$  baseline mode;
7:         return {one task only}
8:       end if
9:     end if
10:  end for
11: end for

```

son between HADS and Burst-HADS, using the source work's own catalogue, job definitions, fixed deadline $D = 2700$ s and scenarios. Table V and Fig. 1 report the outcome.

Our HADS implementation lands within a few percent of the published makespans, in every case pinned just below the deadline, which is the expected behaviour of a cost-minimising greedy that packs to its budget. The direction of the cost difference is reproduced in every job and its magnitude closely in three of four: Burst-HADS is faster than HADS and slightly more expensive, which is what the source work reports.

Our reproduction of the makespan *reduction* is smaller than published, and the shortfall grows with job size. The most plausible explanation is capacity: at a fixed $D = 2700$ s the largest job consumes the whole pool, leaving the search no spare instances to spread onto. We report this as a limitation of the reproduction rather than a correction to the source work.

VII. RESULTS*A. Task completion and deadline satisfaction*

All three schedulers completed 100% of tasks in all 30 configurations, so completion rate does not discriminate between them. Deadline satisfaction does. Burst-HADS and R-BurstHADS met the deadline in every configuration. HADS

TABLE II
VM CATALOGUE. THREE INSTANCES OF EACH SPOT TYPE.

Type	Market	s_j	nvc_j	Mem	\$/h	λ_h
c5.large	spot	2	2	4 GB	0.0306	0.03/3600
c5.xlarge	spot	4	4	8 GB	0.0612	0.04/3600
m5.xlarge	spot	4	4	16 GB	0.0700	5.0/2700
t3.large	burstable	2	1	8 GB	0.0832	—
c5.large	on-demand	2	2	4 GB	0.0850	—

TABLE III
HIBERNATION SCENARIOS

	sc1	sc2	sc3	sc4	sc5
k_h	1	5	1	5	3
k_r	0	0	5	5	2.5

missed in three — sc1, sc2 and sc3 at $|T| = 50$ — the workload size at which the deadline is closest to its feasibility floor and HADS, lacking burstable rescue capacity, must absorb an interruption entirely on on-demand instances subject to start-up latency.

B. Burst-HADS against HADS

At $|T| = 300$ the ordering reported in the source work is reproduced in all five scenarios: Burst-HADS is faster than HADS, by between 1.1% (sc2) and 24.9% (sc3), and more expensive, by between 1.7% (sc4) and 16.1% (sc1). The ordering weakens as the workload shrinks — four of five scenarios at $|T| = 200$, two of five at $|T| = 100$ — because a smaller workload leaves the pool underutilised and the two schedulers converge on similar placements. We therefore treat $|T| \geq 200$ as the regime in which this comparison is informative. Fig. 4 shows the speedup of both extensions relative to HADS, and Fig. 5 the fraction of the deadline each scheduler consumes.

C. R-BurstHADS against Burst-HADS

Fig. 2 shows the full comparison, Fig. 6 summarises R-BurstHADS’s advantage, and Fig. 9 resolves it per cell. Two patterns are visible.

First, the **makespan advantage is large and rate-independent**: R-BurstHADS is 26.0–39.8% faster than Burst-HADS in every scenario, with no systematic relationship to k_h . This is the expected signature of elastic provisioning: whenever spot capacity is lost, replacement capacity is acquired in proportion to the displaced work rather than drawn from a standing pool of fixed size.

Second, the **cost advantage scales monotonically with the interruption rate** (Fig. 7). At $k_h = 1$ the two schedulers are indistinguishable on cost (+0.5% in sc1, −0.5% in sc3); at $k_h = 3$ R-BurstHADS is 12.5% cheaper; at $k_h = 5$ it is 14.3% cheaper when instances resume (sc4) and 64.3% cheaper when they do not (sc2). Under frequent interruption Burst-HADS exhausts its standing burstable capacity early and places the remainder of the displaced work on on-demand instances, the most expensive resource available, whereas R-BurstHADS

TABLE IV
ALGORITHM AND EXPERIMENT PARAMETERS

Parameter	Value
α	0.5
$max_iteration, max_attempt$	200, 50
$swap_rate, max_failed$	0.10, 20
$relaxed_rate$	0.25
$burst_rate$	0.2
Allocation cycle	900 s
T_{start}, ω	45 s, 0.10
ρ, max_per_event	0.05, 10
DF, κ	1.0, 3
Seeds per configuration	30

TABLE V
BASELINE VALIDATION AGAINST PUBLISHED RESULTS

Job	HADS mcp (s)		Δ cost (%)		Δ mcp
	ours	pub.	ours	pub.	
J60	2476	2620	+29.5	+30.8	−33.9
J80	2501	2581	+25.0	+11.3	−36.0
J100	2541	2518	+31.3	—	−8.4
ED200	2694	2680	+21.6	+23.6	−2.7

replaces lost spot capacity with spot capacity. That the cost advantage tracks a parameter which was not tuned, and which controls the environment rather than the algorithm, is the strongest evidence that the effect is mechanistic rather than an artefact of configuration.

Fig. 8 places both extensions in the cost–makespan plane relative to HADS, and Fig. 10 normalises cost per task. Table VI gives the summary figures and Table VII the full grid.

VIII. DISCUSSION

A. When the contribution is inert

R-BurstHADS provisions nothing when the deadline is not threatened, which is a property of (4) rather than an accident. In the validation experiments, where D is fixed at 2700 s against a pool sized for a much shorter makespan, R-BurstHADS is identical to Burst-HADS in most cells. The exception is the largest job, where memory pressure binds and it is 33% cheaper. This is the intended behaviour: the mechanism costs nothing when it is not needed, and its value is confined to deadline-pressured regimes.

B. Cost–makespan trade

Neither extension dominates HADS on cost. HADS is the cheapest of the three in most configurations, because it packs onto the fewest instances and consumes its deadline budget — visible directly in Fig. 5. What the extensions buy is time and deadline safety: HADS is the only scheduler in the study that misses a deadline, and it is 26–40% slower than R-BurstHADS at every workload size above 100 tasks. Which of these is preferable is a property of the application, not of the scheduler, and we report both rather than collapsing them into a single figure of merit.

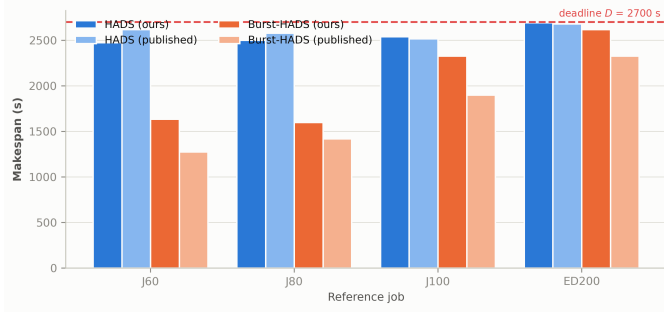


Fig. 1. Validation against the published results. Our HADS lands within a few percent of the reported makespans, pinned just below the deadline.

TABLE VI
R-BURSTHADS RELATIVE TO BURST-HADS, MEAN OVER $|T| \geq 100$

Scenario	k_h	k_r	Cost (%)	Makespan (%)
sc1	1	0	+0.5	+39.8
sc3	1	5	-0.5	+35.6
sc5	3	2.5	+12.5	+29.7
sc4	5	5	+14.3	+28.3
sc2	5	0	+64.3	+26.0

C. Limitations

The evaluation is a simulation and inherits the abstractions of its model. Task execution times are drawn from a uniform distribution whose range matches the reference jobs but whose shape does not reflect the right-skewed, straggler-heavy profile of real workloads; this understates the tail that scheduling must absorb. Hibernation stops compute billing but incurs no storage charge, where the provider does charge for the retained EBS volume; the omitted quantity is small relative to compute but non-zero. The per-instance-type differentiation of hibernation rates, as noted in Section V, is a modelling choice of this study. Finally, the baseline reproduction degrades on the largest job, so conclusions drawn at that scale rest on a weaker validation than those at smaller ones.

IX. CONCLUSION

We presented R-BurstHADS, which extends Burst-HADS with elastic spot provisioning sized against the remaining deadline by a work-conservation argument, drawing from the same instance catalogue available to the baselines. Across 2,700 simulation runs it reduced makespan relative to Burst-HADS by 26.0–39.8% in every hibernation scenario, and its monetary-cost advantage scaled monotonically with the expected interruption count, from cost-neutral at $k_h = 1$ to 64.3% cheaper at $k_h = 5$ without resumption. Both extensions met the deadline in every configuration, where HADS did not.

Two directions follow. The provisioning rule currently treats replacement instances as equally reliable as those they replace; incorporating the replacement’s own interruption risk into (4) would make the sizing risk-aware as well as deadline-aware. Second, the evaluation should move from a synthetic uniform workload to real BoT traces, where the straggler tail is the

dominant source of makespan and the value of elastic rescue capacity is likely to be larger than reported here.

REFERENCES

- [1] L. Teylo, L. Arantes, P. Sens, and L. M. de A. Drummond, “A dynamic task scheduler tolerant to multiple hibernations in cloud environments,” *Cluster Computing*, vol. 24, no. 2, pp. 1051–1073, 2021.
- [2] L. Teylo, L. Arantes, P. Sens, and L. M. de A. Drummond, “Scheduling bag-of-tasks in clouds using spot and burstable virtual machines,” *IEEE Transactions on Cloud Computing*, 2023.
- [3] L. Teylo, L. Arantes, P. Sens, and L. M. de A. Drummond, “A hibernation aware dynamic scheduler for cloud environments,” in *Proc. 48th Int. Conf. on Parallel Processing Workshops (ICPP)*, 2019.
- [4] Amazon Web Services, “Amazon EC2 Spot Instances,” <https://aws.amazon.com/ec2/spot/>.
- [5] Amazon Web Services, “Burstable performance instances,” *Amazon EC2 User Guide*.



Fig. 2. Makespan (top) and monetary cost (bottom) against workload size for each hibernation scenario. Means over 30 seeds. Lower is better.

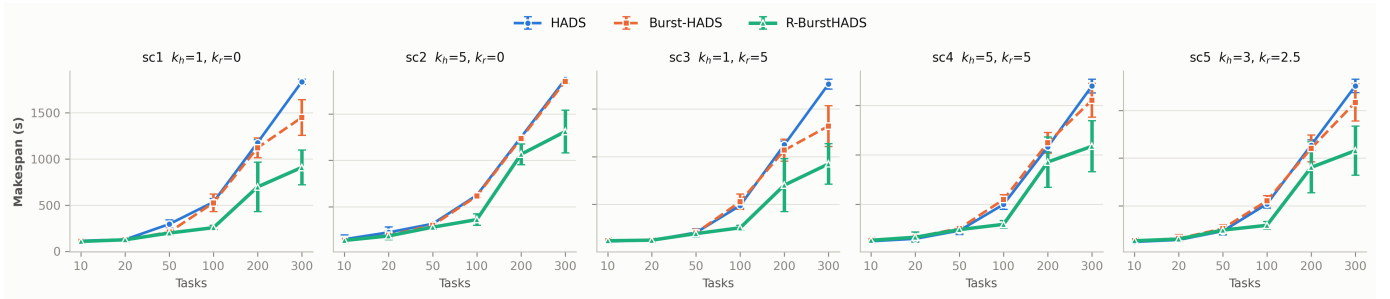


Fig. 3. Makespan with ± 1 standard deviation over the 30 seeds. Dispersion grows with the interruption rate but the ordering is stable.



Fig. 4. Speedup relative to HADS. Values above unity indicate a faster scheduler.



Fig. 5. Deadline utilisation, i.e. makespan divided by D . HADS consumes its budget almost entirely; R-BurstHADS retains headroom.

TABLE VII
FULL RESULTS. MONETARY COST (USD) AND MAKESPAN (s), MEANS OVER 30 SEEDS.

Sc.	n	HADS		Burst-HADS		R-BurstHADS	
		cost	mkp	cost	mkp	cost	mkp
sc1	10	0.0400	114	0.0321	108	0.0317	107
	20	0.0658	128	0.0530	125	0.0530	125
	50	0.1150	295	0.0868	208	0.0831	198
	100	0.1160	533	0.1125	525	0.1230	256
	200	0.1670	1177	0.1938	1121	0.1934	698
	300	0.2141	1836	0.2486	1449	0.2356	908
sc2	10	0.0272	150	0.0190	137	0.0190	137
	20	0.0562	223	0.0388	192	0.0348	185
	50	0.1947	316	0.2043	300	0.0954	279
	100	0.2762	624	0.2643	614	0.0753	361
	200	0.3950	1248	0.4017	1234	0.1563	1065
	300	0.5053	1874	0.5292	1852	0.1945	1310
sc3	10	0.1003	112	0.0937	119	0.0937	119
	20	0.1234	124	0.1128	125	0.1128	125
	50	0.1731	204	0.1443	201	0.1416	194
	100	0.1707	487	0.1771	528	0.1848	254
	200	0.2248	1124	0.2570	1067	0.2592	701
	300	0.2709	1759	0.3072	1320	0.3007	923
sc4	10	0.0953	126	0.0813	136	0.0813	136
	20	0.1044	151	0.0894	168	0.0889	167
	50	0.1684	234	0.1719	256	0.1393	244
	100	0.1884	499	0.1854	548	0.1503	297
	200	0.2615	1080	0.2713	1123	0.2417	928
	300	0.3364	1696	0.3421	1552	0.2929	1087
sc5	10	0.0880	122	0.0762	133	0.0762	133
	20	0.0985	142	0.0873	157	0.0864	151
	50	0.1689	235	0.1700	260	0.1273	245
	100	0.1781	519	0.1715	553	0.1451	295
	200	0.2480	1135	0.2511	1102	0.2313	903
	300	0.3165	1758	0.3255	1585	0.2780	1079

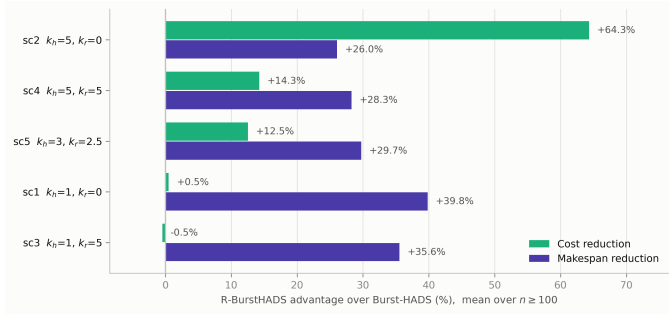


Fig. 6. R-BurstHADS's advantage over Burst-HADS, averaged over $|T| \geq 100$.

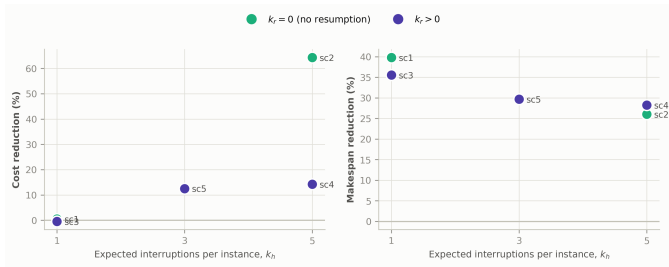


Fig. 7. Advantage against the expected interruption count k_h . The cost advantage is monotone in k_h ; the makespan advantage is not.

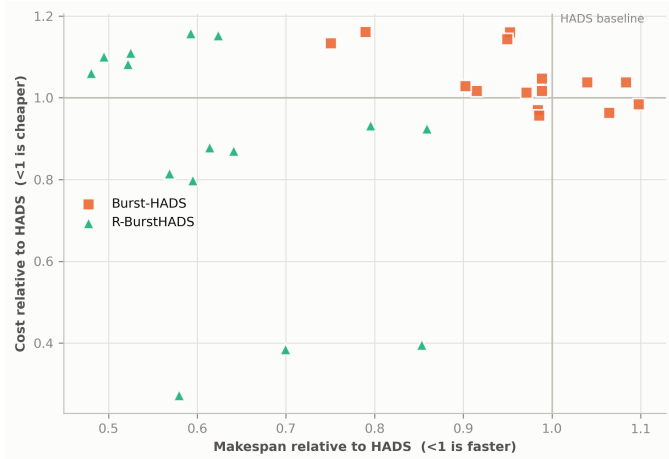


Fig. 8. Cost–makespan plane relative to HADS, for $|T| \geq 100$. R-BurstHADS occupies the faster region; Burst-HADS clusters near the baseline.

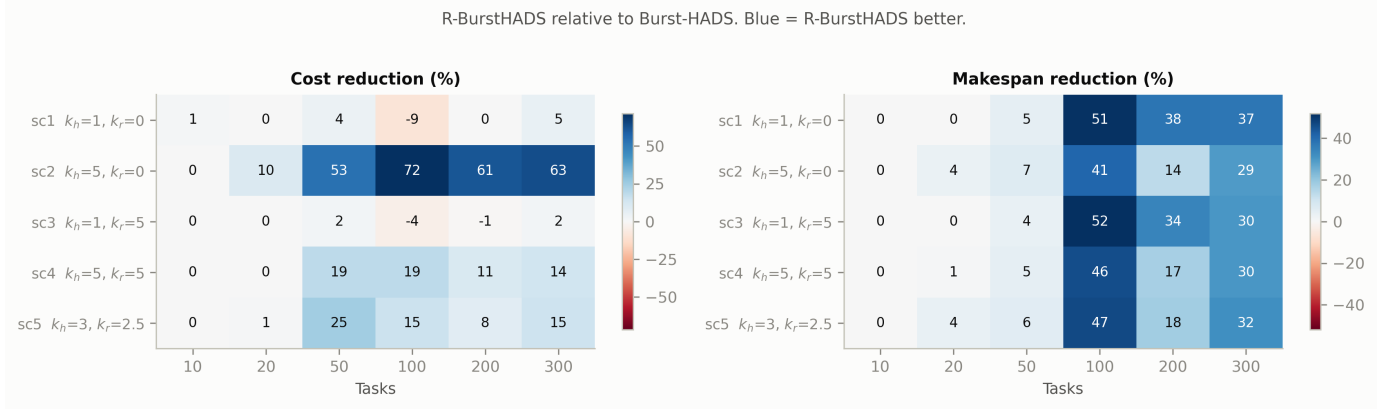


Fig. 9. Per-cell advantage of R-BurstHADS over Burst-HADS.

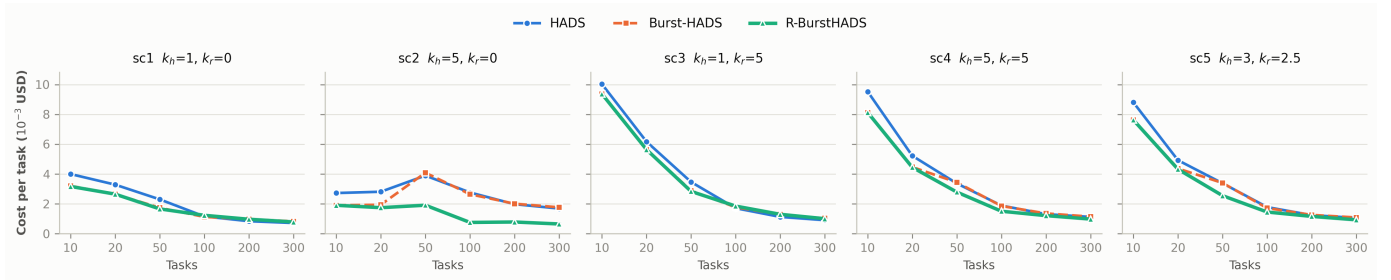


Fig. 10. Cost per task. Normalising by workload size isolates the per-unit efficiency of each scheduler.